



Instituto
de Ciencias
Tecnológicas



Framework de soluciones empresariales

Síntesis de la Unidad 2:

MVC y Framework



Tema General

INTRODUCCIÓN

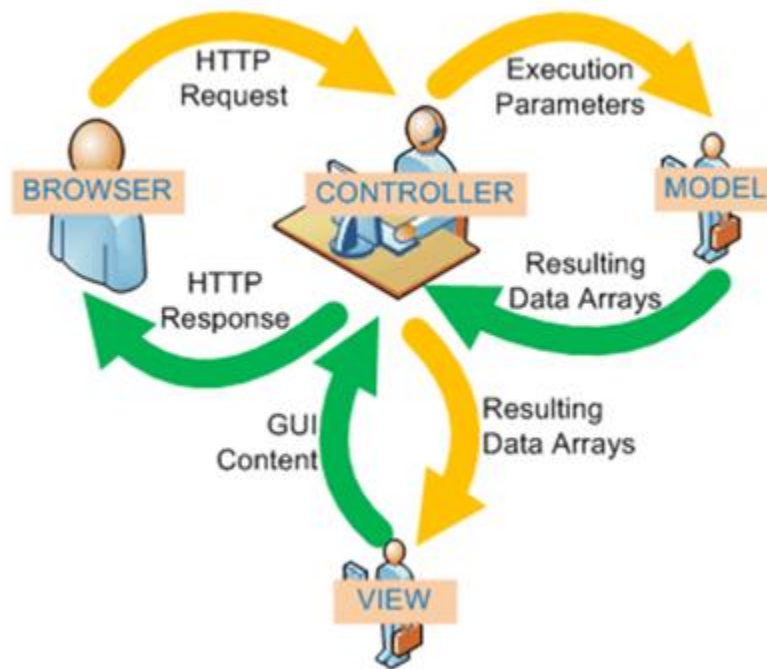
Al momento de buscar desarrollar una aplicación de forma rápida, modular y mantenible es aconsejable implementar un patrón de diseño de software probado como lo es MVC, el cual separa las funciones de la aplicación en modelos, vistas y controladores.

1.MVC

Existe dos marcos marcos arquitectónicos comúnmente utilizados para crear aplicaciones basadas en web: Model-View-Controller y Web-Forms. MVC se conceptualizó originalmente en 1976 como una arquitectura de desarrollo para crear aplicaciones de escritorio.

MVC se ha convertido desde entonces en un marco para crear aplicaciones compatibles multiplataforma usando una variedad de lenguajes de programación que incluyen Ruby on Rails, .NET, Java y muchos otros. Es una arquitectura popular porque reduce la duplicación de tareas al eliminar la necesidad de crear múltiples versiones de la misma aplicación para usar en diferentes plataformas operativas.

MVC se adhiere al principio de Separación de preocupación (o roles) porque permite a un desarrollador dividir una aplicación en secciones en función de cómo la información interactúa con el código. Por ejemplo, separa la interfaz de usuario de una aplicación de su lógica comercial y la implementación de la lógica de acceso a datos a través de las funciones Ver y Controlador, lo que permite que un desarrollador actualice cada sección individualmente.



Referencia: <https://si.ua.es/es/documentacion/asp-net-mvc-3/1-dia/modelo-vista-controlador-mvc.html>

2. Las responsabilidades del rol del Modelo:

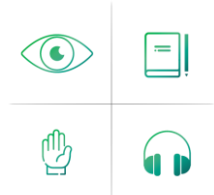
- Acceder a la capa de almacenamiento de datos. Lo ideal es que el modelo sea independiente del sistema de almacenamiento.
- Define las reglas de negocio (la funcionalidad del sistema). Un ejemplo de regla puede ser: "Si la mercancía pedida no está en el almacén, consultar el tiempo de entrega estándar del proveedor".
- Lleva un registro de las vistas y controladores del sistema.
- Si estamos ante un modelo activo, notificará a las vistas los cambios que en los datos pueda producir un agente externo (por ejemplo, un fichero por lotes que actualiza los datos, un temporizador que desencadena una inserción, etc.)

3. Las responsabilidades del rol del Controlador:

- Recibe los eventos de entrada (un clic, un cambio en un campo de texto, etc.).
- Contiene reglas de gestión de eventos, del tipo "SI Evento Z, entonces Acción W". Estas acciones pueden suponer peticiones al modelo o a las vistas. Una de estas peticiones a las vistas puede ser una llamada al método "Actualizar ()". Una petición al modelo puede ser "Obtener_tiempo_de_entrega (nueva_orden_de_venta)".

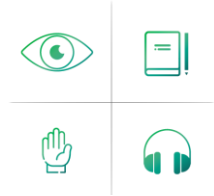
4. Las responsabilidades del rol del rol Vista:

- Recibir datos del modelo y los muestra al usuario.
- Tienen un registro de su controlador asociado (normalmente porque además lo instancia).
- Pueden dar el servicio de "Actualización ()", para que sea invocado por el controlador o por el modelo (cuando es un modelo activo que informa de los cambios en los datos producidos por otros agentes).



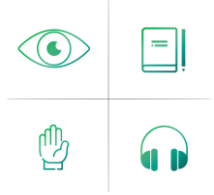
La arquitectura MVC divide las capas en:

1. Roles o Responsabilidades. (Correcta)
2. Objetivos o funcionalidades.
3. Tipos de archivos.



Es el encargado de recibir los eventos de entrada, nos referimos a:

1. Controlador (Correcta)
2. Modelo
3. Vista



Es el encargado de mostrar datos al usuario, nos referimos a:

1. Vista (Correcta).
2. Modelo.
3. Controlador.



Es el encargado de acceder a la capa de almacenamiento de datos.

4. Modelo (Correcta)
5. Controlador.
6. Vista.

Conclusión

Podemos concluir que el uso de mvc en el desarrollo de software como patrón de arquitectura es la separación del modelo, vista y controlador, lo cual logra separar los datos, de su representación visual, facilita el manejo de errores, permite que el sistema sea escalable si es requerido, y es posible agregar múltiples representaciones de los datos, por otro lado también podemos ver desventajas, como la cantidad de archivos que se deben mantener se incrementan, es mas complejo de entender e implementar que otros modelos y al separar por capas aumenta la complejidad, pero al analizar y decidir, las ventajas son razones mas poderosas que las desventajas, por lo cual hace ser que MVC sea ampliamente validado y utilizado en el desarrollo de software.

Bibliografía

Referencia del presente documento:

<https://si.ua.es/es/documentacion/asp-net-mvc-3/1-dia/modelo-vista-controlador-mvc.html>